

OOPs Concepts in Python

Are you ready to take your Python programming skills to the next level? Today, I want to talk about one of the fundamental aspects of Python: Object-Oriented Programming (OOPs). Explore Python OOPs concepts effortlessly.

“Object-oriented programming offers a sustainable way to write spaghetti code.” — Paul Graham

Object-oriented programming (OOP) is a programming paradigm concepts that is based on the concept of “objects”, which can contain data and code that manipulates that data.

There are 4 OOP concepts. They are:

1. Polymorphism

2. Inheritance

3. Encapsulation

4. Abstraction



Python's OOPs allows us to organize our code into reusable, modular, and efficient structures. By understanding and leveraging Python OOPs concepts, we can create robust and scalable applications that are easier to maintain and extend.

Let's dive into some key concepts!

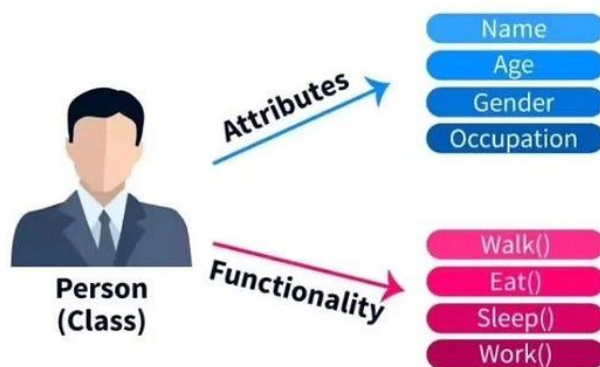
Classes

At the heart of OOPs in Python concepts, we have classes. A class is a blueprint for creating objects, defining their attributes (data) and methods (functions). It encapsulates related data and behavior, providing a clear structure to work with.

CLASS

A class is a blueprint for declaring and creating objects.

What is Class?

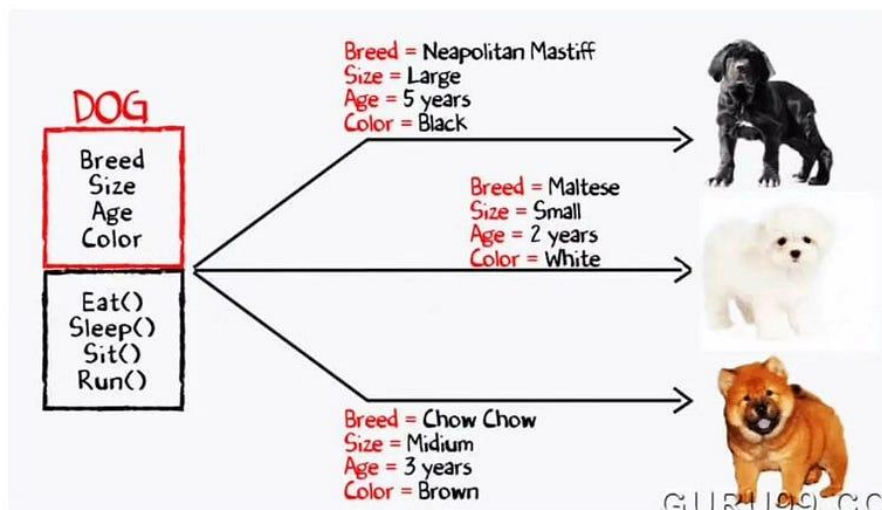


Objects

Objects are instances of a class. They are created based on the class blueprint and can have their own unique data and behavior. By using objects, we can create multiple instances that share the same attributes and methods defined in the class.

OBJECT

An object is a class instance that allows programmers to use variables and methods from inside the class.



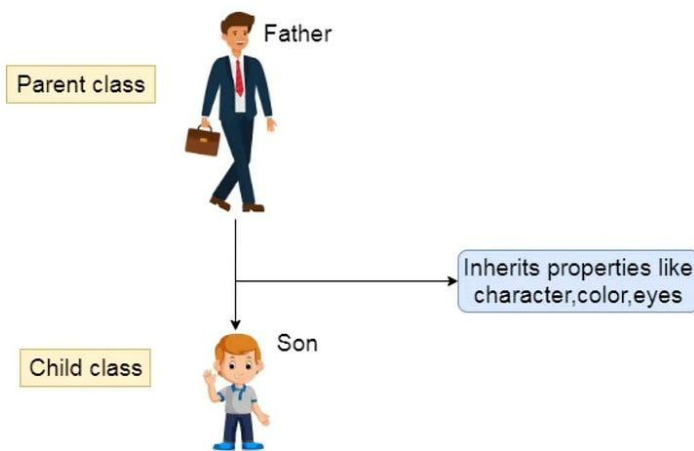
@Python.hub

Inheritance

Inheritance allows us to create new classes based on existing ones. It promotes code reuse and hierarchy, enabling us to define general characteristics in a base class and extend or modify them in derived classes. In Python, we can inherit from multiple classes, making it highly flexible.

INHERITANCE

Inheritance means it allows classes to inherit common properties from the parent class.

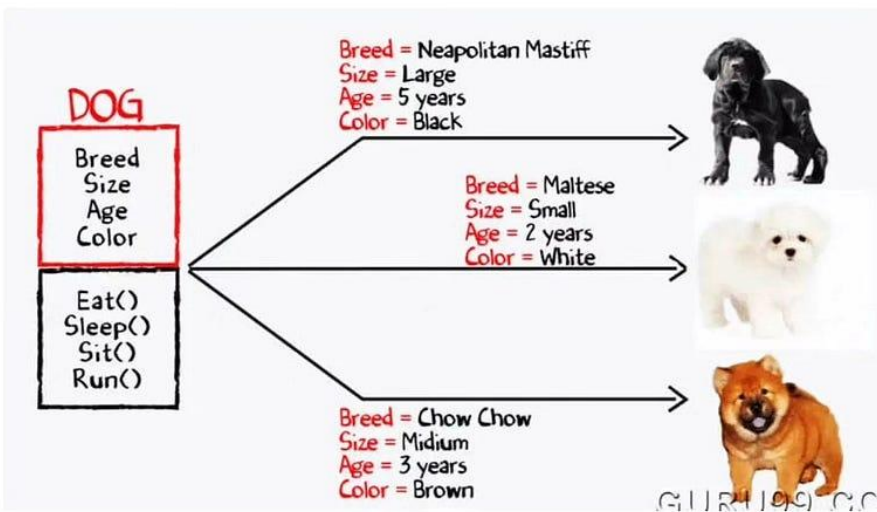


Encapsulation

Encapsulation refers to the bundling of data and methods within a class. It allows us to control access to class members, making them private or public. This principle enhances data security, maintains code integrity, and reduces dependencies.

OBJECT

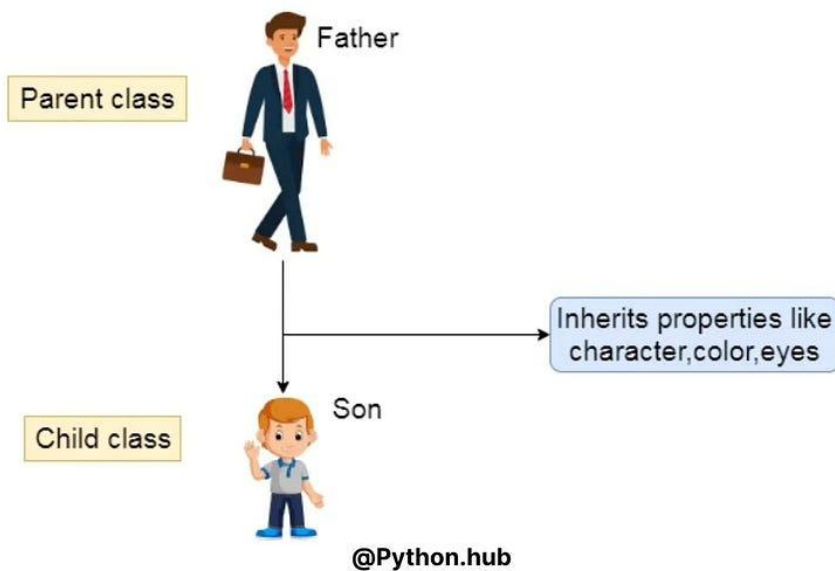
An object is a class instance that allows programmers to use variables and methods from inside the class.



- 5) **Polymorphism:** Polymorphism is the ability of objects to take on many forms. It allows us to define methods in different classes with the same name but different implementations. Polymorphism promotes code flexibility, as objects can be used interchangeably even if they belong to different classes.

INHERITANCE

Inheritance means it allows classes to inherit common properties from the parent class.



Abstraction

Abstraction focuses on providing simplified interfaces while hiding complex underlying implementations. By defining abstract classes and methods, we can enforce consistent behavior across subclasses while allowing specific implementations to be developed separately.

ABSTRACTION

In abstraction, it displays only the important information by hiding the implementation part.



By leveraging these OOPs concepts, Python developers can write modular, reusable, and scalable code. Whether you're building web applications, data analysis tools, or even game development projects, OOP in Python will be your trusty companion.

Class and Object

A class is a blueprint for creating objects (a particular data structure), providing initial values for state (member variables or attributes), and implementations of behavior (member functions or methods). A class can also define class variables and class methods, which are shared by all instances of the class.

In Python, classes are defined using the `class` keyword and a basic class definition might look like this:

```
class MyClass:
    x = 5

    def print_x(self):
```

```
print(self.x)
```

Python

This class, called MyClass, defines a class variable x with the value of 5, and a class method print_x() which will print the value of x when called.

Creating an object, or an instance of a class, is done by calling the class name as if it were a function:

```
obj = MyClass()
```

Python

This will create an instance of the MyClass class, which we can then use to access the class's attributes and methods:

```
obj.x = 10
obj.print_x() # prints 10
```

Python

Objects of the same class are created independently of each other and have their own property values.

```
object1 = MyClass()
object2 = MyClass()
object1.x = 15
print(object2.x) # prints 5
```

Python

To put it simply, a class is like a blueprint of a house and an object is like a house built from that blueprint. The blueprint (class) can be used to build any number of houses (objects), but each house will have its own unique characteristics.

In Python, Classes also support the following Features

- Constructors, special methods that are called when an object is created, using `__init__` method.
- Special class-level methods and variables, like `__str__` and `__doc__`
- The ability to inherit from other classes, which allows for code reuse and simplification of object-oriented design.
- Encapsulation, the ability to hide the implementation details of an object from other parts of the program, and providing a public interface to interact with the object.
- Polymorphism, the ability for objects of different classes to be treated as objects of a common class, allowing for more flexible and reusable code.

These features, when used correctly, can make Python classes a powerful tool for creating elegant, dynamic, and robust code. They provide a way to organize and structure code and make it easier to maintain and extend over time.

Inheritance

Inheritance is a powerful feature of object-oriented programming that allows a class to inherit the properties and methods of another class. This allows for a natural organization of code and can also help to reduce code duplication.

In Python OOPs concepts, a class can inherit from another class by specifying the parent class in parentheses when defining the class. For example:

```
class Shape:
    def area(self):
        pass
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
    def perimeter(self):
        return 2 * (self.width + self.height)
```

Python

Here, the Rectangle class inherits from the Shape class, which has methods for calculating the area and perimeter. The Rectangle class can use these methods as-is, or override them if needed to provide its own implementation.

Python also supports multiple inheritance, which allows a class to inherit from multiple classes. For example:

```
class Square(Rectangle, Shape):
    def __init__(self, side):
        self.width = side
        self.height = side
```

Python

Here, the Square class inherits from both the Rectangle and Shape classes. This allows the Square class to inherit properties and methods from both parent classes.

When a class inherits from multiple classes that define a method with the same name, python uses a method resolution order (MRO) algorithm to determine which method should be called. The C3 algorithm is the most common MRO in python, this algorithm is used to resolve the order of method calls in the case of multiple inheritance and ensure that the order is predictable and efficient.

Multiple inheritance can be a powerful tool, but it can also make code more complex and harder to understand. As it can be tricky to understand the method resolution order. Thus it is important to use it judiciously and only when it is actually needed.

Also, when a class has circular inheritances or diamond-shaped inheritances, it can lead to confusion and unexpected results. In such scenarios, it's better to use composition to avoid such complexities.

Encapsulation

Encapsulation is a fundamental concept in object-oriented programming that refers to the practice of hiding the internal implementation details of an object from other parts of the

program. This is achieved by defining class methods and variables as private and providing public methods for interacting with them.

In Python, encapsulation is achieved by using the `_` prefix for member variables and methods. For example, if we have a `Person` class:

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age

    def _get_age(self):
        return self._age
```

Python

Here, the `_name` and `_age` attributes are considered private and the `_get_age()` method is private as well. These are intended to be used only within the class, and not directly accessed or modified by code outside of the class.

Instead, we would provide public methods for interacting with these private variables and methods, like this:

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age

    def _get_age(self):
        return self._age

    def get_age(self):
        return self._get_age()
```

Python

Here, `get_age` is a public method.

Encapsulation has several benefits in Python:

1. It allows for code modularity, which makes it easier to change and maintain the codebase.
2. It ensures that the internal state of an object is only modified in a controlled way, which helps to prevent bugs and maintain consistency.
3. It makes the code more robust and secure since it is more difficult for other parts of the program to accidentally or maliciously change the internal state of an object.
4. It makes the code more flexible since the internal implementation can be changed without affecting the code that uses the class.

However, it's important to note that, in python, encapsulation is not enforced by the interpreter and relies on the programmer to not access private variables or methods directly. It's more of a convention and a way to signal to others that these methods or attributes are intended to be used only by the class.

Polymorphism

Polymorphism is a fundamental concept in object-oriented programming that allows objects of different classes to be treated as objects of a common class. In other words, it allows an object to take on many forms. This can make code more flexible and reusable.

In Python OOPs concepts, polymorphism is achieved through polymorphic functions, which are functions that can work with multiple types of input. For example, the `len()` function can be used to find the length of a string, a list, or other types of data:

```
>>> len("hello")
5
>>> len([1, 2, 3])
3
```

Python

Another example is the `print()` function which can take an arbitrary number of arguments and print each one:

```
>>> print("Hello", "World!")
Hello World!
>>> print("The value of x is:", x)
The value of x is: 10
```

Python

Python also supports polymorphism through method overriding and method overloading.

Method overriding occurs when a subclass provides a different implementation of a method that is already defined in the parent class. This allows for objects of the subclass to have a different behavior than objects of the parent class.

For example:

```
class Shape:
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * (self.radius ** 2)
```

Python

Here, both `Rectangle` and `Circle` class inherits from the `Shape` class and have different implementations for `area` method. When `area` is called on an instance of `Rectangle` and `Circle` class, it will call the overridden method.

Method overloading is not supported in python. But, we can achieve the same behavior with methods that take different arguments and have different behavior depending on the type of argument. By using polymorphic functions, method overriding, and method overloading, Python allows developers to create more expressive and efficient code.

First, what is a method?

A method is just a function that lives inside a class. It helps objects do things!

1. **Method Overloading** (Not directly supported like other languages)

Imagine you have a robot toy that can say hello in different ways:

Say hello with just your name.

Say hello with your name and age.

Say hello with your name, age, and city.

You might want to write different versions of the same function (say_hello) to do this.

In Python, we can't have multiple methods with the same name but different parameters (like in Java or C++), but we can still make it work using default arguments or *args.

Example:

```
class Robot:
    def say_hello(self, name=None, age=None):
        if name and age:
            print(f"Hello, I am {name} and I am {age} years old!")
        elif name:
            print(f"Hello, I am {name}!")
        else:
            print("Hello, I am a robot!")

r1 = Robot()
r1.say_hello()
r1.say_hello("Tom")
r1.say_hello("Tom", 10)
```

This is how Python pretends to do method overloading!

2. **Method Overriding** (Fully supported)

Now, imagine you have a Parent class and a Child class.

Let's say the Parent class has a method called greet(), and the Child class wants to use its own version of greet().

Example:

```
class Animal:
    def sound(self):
        print("Some animal sound")

class Dog(Animal):
    def sound(self):
        print("Woof woof!")

a1 = Animal()
a1.sound()  # Output: Some animal sound

d1 = Dog()
d1.sound()  # Output: Woof woof!
```

This is method overriding – the child class replaces the parent class method with its own version!

Concept:

Method Overloading Same method name, different behavior using different arguments (tricked in Python using default args or *args)

Method Overriding Child class replaces parent class method with its own version

Data Abstraction

Data abstraction is a fundamental concept in object-oriented programming that refers to the practice of hiding the implementation details of an object from other parts of the program and providing only a simplified public interface to interact with the object. This allows for a separation between the implementation of an object and the way it is used, making the code more modular, and easier to maintain.

In Python OOPs concepts, data abstraction is achieved through the use of abstract classes and interfaces. An abstract class is a class that defines one or more abstract methods, which are methods that have no implementation. A subclass is required to implement these methods before an instance of the class can be created.

For example:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
    def perimeter(self):
        return 2 * (self.width + self.height)
```

Python

Here, the Shape class is an abstract class, which has two abstract methods, area and perimeter. The subclass Rectangle implements these methods before an instance of the class can be created.

An interface in python is just an abstract class with no implementations for any of its methods. Python has no inbuilt support for interface but we can use ABC(Abstract base class) from python's abc module as an interface.

For example:

```
from abc import ABC

class Myinterface(ABC):
    @abstractmethod
    def method1(self):
        pass

    @abstractmethod
    def method2(self):
        pass
```

Python

This interface `Myinterface` has two abstract methods, `method1` and `method2`. Any class that wants to implement this interface, it needs to implement these methods.

Data abstraction can be used to encapsulate complex code, hide implementation details, and make the code more modular and easier to maintain. By using abstract classes and interfaces, developers can create more expressive and efficient code in python.

The Advantages of OOP in Python

1. It helps to organize code by breaking it down into smaller, reusable pieces.
2. It makes it easy to model real-world objects and their behavior.
3. It promotes code reuse through inheritance.
4. It provides a clear way to represent the relationships between different objects and their properties and methods.

The disadvantages of OOP in Python

1. It can be more difficult to understand how an object-oriented program works as compared to a procedural program.
2. Programs that are heavily object-oriented can have a lot of classes and can become hard to navigate.
3. Python OOPs concepts can lead to more complex programs and sometimes can increase the difficulty of debugging.

Conclusion

Object-Oriented Programming (OOPs) concepts is a powerful paradigm that revolutionizes software development by providing a structured approach to modeling real-world entities and their interactions. By emphasizing principles such as encapsulation, inheritance, polymorphism, and abstraction, OOP enables developers to create modular, reusable, and maintainable code. Through the use of classes, objects, and the implementation of python OOPs concepts, programmers can build sophisticated software systems that accurately reflect the complexities of the real world. Understanding and mastering OOP principles are essential skills for modern software developers, empowering them to design elegant solutions to complex problems and drive innovation in the field of computer science.

Frequently Asked Questions

Q1. What is Object-Oriented Programming (OOP)?

Q2. What are classes and objects in OOP?

Q3. What is the difference between inheritance and composition in OOP?

Q4. Why is OOP important in software development?